



**Course Title: Data Structure**

**Submitted To:**

*Mam Asra Nadeem*

**Group Members:**

*Fatima Tehreem (242201073)*

*Alina Zia(242201087)*

*Esha Masood(242201062)*

**Project Title**

*Train Reservation System*



# Project Proposal: Train Reservation System

## 1. Introduction / Problem Statement

Booking train tickets manually has always been a tedious and error-prone process. Passengers often face long queues at railway stations, difficulty in knowing seat availability, and problems when trying to cancel or change tickets. Railway staff also spend a lot of time managing bookings, maintaining passenger records, and updating seat availability.

A Train Reservation System (TRS) is designed to automate this entire process. It provides a user-friendly interface for passengers to book, cancel, and check tickets, while railway authorities can efficiently manage trains, schedules, and passenger records. By digitizing ticket booking, the system reduces manual errors, saves time, and improves customer satisfaction.

## Objectives / Scope of the Project

### Objectives:

- To develop a computerized system for booking and cancelling train tickets efficiently.
- To allow users to check real-time seat availability for different trains and classes.
- To store and maintain passenger details such as name, age, gender, and contact information.
- To generate unique PNR numbers for every booking for easy tracking.
- To produce reports for booked tickets, available seats, and train schedules.
- To implement a system that can handle waiting lists automatically when trains are full

### Scope:

- Passengers can book and cancel tickets online or through a console-based application.
- Railway staff can update train schedules, total seats, and manage passenger information.
- The system can handle multiple trains and different classes (e.g., AC, Sleeper, General).
- Generates alerts for fully booked trains and waiting list confirmations.

## Methodology

1. **Train Initialization:** Enter train details (number, name, schedule, total seats) and store them in arrays or objects.
2. **Ticket Booking:** Check seat availability, input passenger details, allocate seat, generate PNR, and update records.
3. **Ticket Cancellation:** Input PNR or passenger details, free the seat, update waiting list and records.
4. **Display Information:** Show trains, schedules, available seats, and passenger booking details.
5. **Waiting List Management:** Automatically add passengers to waiting list when trains are full and allocate seats when available.

## **Expected Output**

- Display of all trains along with available seats and class-wise capacity.
- Confirmation of successful ticket bookings with PNR number.
- Real-time seat updates after bookings or cancellations.
- Automatic management of waiting lists and notifications for available seats.
- Display of passenger lists for trains and administrative reports for railway authorities.
- Error messages for invalid inputs or full trains.

## **Documentation**

### **Train Reservation System using Data Structures**

#### **1. Introduction**

This project is a Train Reservation System developed using C++.

The system demonstrates the practical use of Data Structures such as Binary Search Tree (BST), Linked List, Queue, and Stack to manage train search, ticket booking, waiting list, and ticket cancellation efficiently.

#### **2. Objectives of the Project**

- To manage train information efficiently
- To allow passengers to book and cancel tickets
- To handle limited seats using a waiting list
- To implement real-life problems using core data structures

#### **3. Data Structures Used**

##### **3.1 Binary Search Tree (BST)**

- Used to store train information
- Each node contains:
  - Train Number
  - Train Name
- BST allows fast searching of trains based on train number

#### **Operations implemented:**

- Insert Train
- Search Train

##### **3.2 Linked List**

- Used to store confirmed passengers
- Each node contains:
  - Ticket Number
  - Passenger Name
- Linked list allows dynamic insertion and deletion of passengers

#### **Operations implemented:**

- Add Passenger
- Display Passengers
- Remove Passenger on ticket cancellation

### **3.3 Queue**

- Used for Waiting List
- When all seats are full, passengers are added to the queue
- Queue follows FIFO (First In First Out) principle

### **Operations implemented:**

- Add passenger to waiting list
- Remove passenger from waiting list when a seat becomes available

### **3.4 Stack**

- Used to store cancelled ticket numbers
- Stack follows LIFO (Last In First Out) principle

### **Purpose:**

- Keeps record of cancelled tickets

## **4. Working of the System**

### **1. Train Search**

- Trains are stored in a BST
- User can search a train using train number

### **2. Ticket Booking**

- If seats are available, passenger is added to confirmed list
- If seats are full, passenger is added to waiting queue

### **3. Ticket Cancellation**

- Passenger is removed from confirmed list
- Ticket number is pushed into cancellation stack
- First passenger from waiting list is moved to confirmed list

### **4. Display Passengers**

- Displays all confirmed passengers with ticket numbers

## **5. Tools & Technologies Used**

- Programming Language: C++
- Compiler: Any standard C++ compiler
- Concepts Used:
  - Pointers
  - Structures
  - Dynamic Memory Allocation
  - Data Structures

## **6. Conclusion**

This project successfully demonstrates how different data structures can be combined to solve a real-world problem efficiently.

The Train Reservation System provides a clear understanding of BST, Linked List, Queue, and Stack operations in C++ and improves problem-solving skills related to data management.

## Source Code

```
#include <iostream>
#include <string>
using namespace std;

// =====
// TRAIN TREE (BST)
// =====

struct Train {
    int trainNo;
    string trainName;
    Train* left;
    Train* right;
};

Train* trainRoot = NULL;

Train* createTrain(int no, string name) {
    Train* t = new Train;
    t->trainNo = no;
    t->trainName = name;
    t->left = t->right = NULL;
    return t;
}

Train* insertTrain(Train* root, int no, string name) {
    if (root == NULL)
        return createTrain(no, name);

    if (no < root->trainNo)
        root->left = insertTrain(root->left, no, name);
    else
        root->right = insertTrain(root->right, no, name);

    return root;
}

bool searchTrain(Train* root, int no) {
    if (root == NULL)
        return false;

    if (root->trainNo == no) {
        cout << "Train Found!" << endl;
        cout << "Train No: " << root->trainNo << endl;
        cout << "Train Name: " << root->trainName << endl;
        return true;
    }
}
```

```

if (no < root->trainNo)
    return searchTrain(root->left, no);
else
    return searchTrain(root->right, no);
}

```

```

// =====
// PASSENGER LINKED LIST
// =====

```

```

struct Passenger {
    int ticketNo;
    string name;
    Passenger* next;
};

```

```

Passenger* head = NULL;
int ticketCounter = 1000;
const int MAX_SEATS = 3;

```

```

int countPassengers() {
    int count = 0;
    Passenger* temp = head;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}

```

```

void addPassenger(string name) {
    Passenger* p = new Passenger;
    p->ticketNo = ticketCounter++;
    p->name = name;
    p->next = head;
    head = p;

    cout << "Ticket Booked Successfully!" << endl;
    cout << "Ticket No: " << p->ticketNo << endl;
}

```

```

void displayPassengers() {
    Passenger* temp = head;
    if (temp == NULL) {
        cout << "No confirmed passengers." << endl;
        return;
    }
    cout << "Confirmed Passengers:" << endl;
    while (temp != NULL) {
        cout << "Ticket No: " << temp->ticketNo << " | Name: " << temp->name << endl;
        temp = temp->next;
    }
}

```

```

// =====
// WAITING LIST QUEUE
// =====
struct QueueNode {
    string name;
    QueueNode* next;
};

QueueNode* front = NULL;
QueueNode* rear = NULL;

void addToWaiting(string name) {
    QueueNode* q = new QueueNode;
    q->name = name;
    q->next = NULL;

    if (rear == NULL)
        front = rear = q;
    else {
        rear->next = q;
        rear = q;
    }

    cout << "All seats full. Added to waiting list." << endl;
}

string removeFromWaiting() {
    if (front == NULL)
        return "";

    QueueNode* temp = front;
    string name = temp->name;
    front = front->next;

    if (front == NULL)
        rear = NULL;

    delete temp;
    return name;
}

// =====
// CANCELLATION STACK
// =====
struct StackNode {
    int ticketNo;
    StackNode* next;
};

```

```
StackNode* top = NULL;
```

```
void pushCancel(int ticketNo) {  
    StackNode* s = new StackNode;  
    s->ticketNo = ticketNo;  
    s->next = top;  
    top = s;  
}
```

```
void cancelTicket(int ticketNo) {  
    Passenger* temp = head;  
    Passenger* prev = NULL;
```

```
    while (temp != NULL && temp->ticketNo != ticketNo) {  
        prev = temp;  
        temp = temp->next;  
    }
```

```
    if (temp == NULL) {  
        cout << "Ticket not found." << endl;  
        return;  
    }
```

```
    if (prev == NULL)  
        head = temp->next;  
    else  
        prev->next = temp->next;
```

```
    pushCancel(ticketNo);  
    delete temp;
```

```
    cout << "Ticket Cancelled Successfully." << endl;
```

```
    if (front != NULL) {  
        string name = removeFromWaiting();  
        addPassenger(name);  
        cout << "Waiting passenger moved to confirmed list." << endl;  
    }  
}
```

```
// =====  
// MAIN FUNCTION  
// =====
```

```
int main() {  
    int choice;
```

```
    // Insert trains into BST  
    trainRoot = insertTrain(trainRoot, 101, "Green Line");  
    trainRoot = insertTrain(trainRoot, 102, "Karakoram Express");  
    trainRoot = insertTrain(trainRoot, 103, "Tezgam Express");
```

```

do {
    cout << "\n===== Train Reservation System =====" << endl;
    cout << "1. Search Train" << endl;
    cout << "2. Book Ticket" << endl;
    cout << "3. Cancel Ticket" << endl;
    cout << "4. Display Passengers" << endl;
    cout << "5. Exit" << endl;
    cout << "Enter choice: ";
    cin >> choice;

    if (choice == 1) {
        int no;
        cout << "Enter Train No: ";
        cin >> no;
        if (!searchTrain(trainRoot, no))
            cout << "Train not found in system." << endl;
    }
    else if (choice == 2) {
        string name;
        cout << "Enter Passenger Name: ";
        cin >> name;

        if (countPassengers() < MAX_SEATS)
            addPassenger(name);
        else
            addToWaiting(name);
    }
    else if (choice == 3) {
        int t;
        cout << "Enter Ticket No: ";
        cin >> t;
        cancelTicket(t);
    }
    else if (choice == 4) {
        displayPassengers();
    }
    else if (choice == 5) {
        cout << "Exiting system..." << endl;
    }
    else {
        cout << "Invalid choice." << endl;
    }
} while (choice != 5);

return 0;
}

```

## **Conclusion**

This project successfully implements a Train Reservation System using fundamental Data Structures in C++.

The system efficiently handles train searching using Binary Search Tree, passenger management using Linked List, waiting list using Queue, and ticket cancellation records using Stack.

Through this project, we gained practical experience in applying data structures to solve real-world problems and improved our understanding of dynamic memory allocation and data handling in C++.